Laboratory 1:
Signals and Systems
Introduction to Continuous-Time and Discrete-Time Signals
Warm-Up: Arduino Signal Generator

Contents

1 Abstract **3**

2 Learning Objectives **3**

3 Theoretical Background **3**
- 3.1 PWM-to-Analog Filtering Architecture . . . 3
- 3.2 Digital Discretization and The Sampling Frequency Constraint . . . 4

4 Pre-Lab Assignment **4**

5 Equipment & Setup **5**

6 Laboratory Procedure **6**
- 6.1 Phase 1: Hardware Assembly and Basic Code Verification . . . 6
  - 6.1.1 Step 1.1: Design and Assemble RC Filter . . . 6
  - 6.1.2 Step 1.2: Baseband Verification Test . . . 6
- 6.2 Phase 2: Signal Acquisition and MATLAB Integration . . . 7
  - 6.2.1 Step 2.1: Waveform Generation . . . 7
  - 6.2.2 Step 2.2: Oscilloscope Capture and Verification . . . 8
  - 6.2.3 Step 2.3: MATLAB Serial Data Acquisition . . . 9
  - 6.2.4 Step 2.4: Data Visualization . . . 11
- 6.3 Phase 3 (The Challenge): Interrupt Optimization . . . 13
  - 6.3.1 Step 3.1: Software-Based (Baseline) Assessment . . . 13
  - 6.3.2 Step 3.2: Hardware Interrupt Refactoring . . . 14
  - 6.3.3 Step 3.3: Sampling Rate Augmentation . . . 17

7 Data Analysis **18**
- 7.1 Analysis Step 1: Amplitude Verification . . . 18
- 7.2 Analysis Step 2: Frequency Verification . . . 18
- 7.3 Analysis Step 3: Jitter Impact Assessment . . . 18
- 7.4 Analysis Step 4: Digital-Analog Mapping Proof . . . 18
- 7.5 Analysis Step 5: Theoretical vs. Systemic Losses Summary Table . . . 18

8 Grading Rubric **19**

Note on Prompts

Throughout this and future lab documents, there will be four different prompts to let you know specific work that needs to be done.

- **Pre-lab Deliverable (N):** Work to be documented in the pre-laboratory assignment.
- **Checkpoint (N):** Work to be shown to a TA during the laboratory.
- **Discussion (N):** Questions to be discussed with your lab partner and then answered in the post-laboratory assignment.
- **Deliverable (N):** Other items to be completed individually for the post-laboratory assignment.

1 Abstract

This laboratory establishes the foundational interface between Continuous-Time (CT) theoretical modeling and Discrete-Time (DT) physical implementation. Utilizing an embedded Arduino platform as an arbitrary waveform generator, students synthesize standard signal topologies (sine, square, triangle) through Pulse Width Modulation (PWM) and Digital-to-Analog Conversion (DAC). Signal acquisition operates on a bifurcated pipeline: analog verification via a digital oscilloscope, and digital serialization into MATLAB for array-based processing. By actively mapping hardware-level captured phenomena to computational constructs, this manual standardizes the continuous-to-discrete data pipeline utilized throughout the remainder of the course.

2 Learning Objectives

1. **CT/DT Waveform Synthesis Formulation:** Map continuous-time analytic equations into discrete-time sample structures embedded on a digital microcontroller utilizing DAC and PWM mechanisms.
2. **Analog Reconstruction Filtering:** Implement a physical passive first-order RC low-pass filter to reconstruct continuous analog envelopes from high-frequency PWM digital carrier signals.
3. **Data Acquisition (DAQ) Infrastructure pipelines:** Validate continuous data serialization by comparing high-fidelity continuous scope traces against discrete time-series vectors acquired and plotted via MATLAB.
4. **Aliasing and Jitter Attenuation:** Investigate the real-time implications of software loop-timing variance (jitter) versus precise hardware-timer interrupts on sampling interval integrity (T_s).

3 Theoretical Background

Digital Signal Processing dictates the transformation of continuous-time events $x(t)$ into discrete-time sample arrays $x[n]$. A microcontroller operates inherently in discrete time governed by a local clock. Therefore, waveform synthesis relies upon discretizing mathematical relationships bounded by fundamental constraints such as the Nyquist-Shannon Sampling Theorem.

3.1 PWM-to-Analog Filtering Architecture

Pulse Width Modulation (PWM) encodes arbitrary analog voltages by dynamically modifying the duty cycle D of a digital pulse train oscillating at a fixed carrier frequency f_{pwm}. The logic-high width T_H characterizes the instantaneous equivalent continuous voltage:

$$v_{out}(t) = \left(\frac{T_H}{T_{pwm}} \right) V_{ref} = D \cdot V_{ref} \tag{1}$$

Because the signal contains large spectral components localized around f_{pwm} and its harmonics, a passive anti-imaging low-pass filter is required to extract the pure baseband continuous signal. The fundamental cutoff frequency f_c of a first-order RC filter is defined as:

$$f_c = \frac{1}{2\pi RC} \tag{2}$$

The parameter space limits f_c to be sufficiently larger than the baseband signal frequency f_0 yet far attenuating compared to the carrier frequency f_{pwm} ($f_0 < f_c \ll f_{pwm}$).

3.2 Digital Discretization and The Sampling Frequency Constraint

To prevent spectral folding (aliasing), the discrete sampling interval T_s must satisfy the continuous counterpart boundary conditions:

$$x[n] = x(nT_s) \Rightarrow f_s = \frac{1}{T_s} \geq 2f_{max} \tag{3}$$

For internal memory look-up-tables within the microcontroller, T_s is controlled via processor loops or dedicated hardware timer interrupts. Interrupt-driven updates limit systemic loop-delay variance (jitter) compared to sequential polling methodologies.

4 Pre-Lab Assignment

Provide rigorous mathematical derivation for the following problems.

1. **Pre-lab Deliverable (1/3):** Filter Design Parameters:Select commercially available R and C for $f_c \approx 50$ Hz using $f_c = \frac{1}{2\pi RC}$.
2. **Pre-lab Deliverable (2/3):** Impulse Response:Derive $h(t)$ and calculate time constant $\tau = RC$ for 63% settling.
3. **Pre-lab Deliverable (3/3):** Quantization Analysis:Calculate quantization voltage step ΔV and SQNR (dB) for 8-bit PWM.

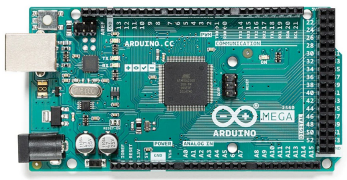
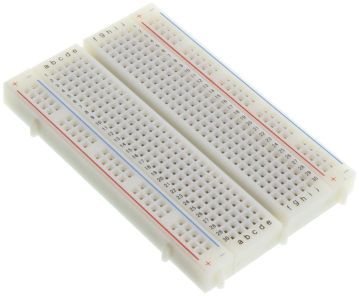
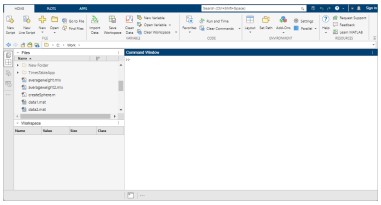
5 Equipment & Setup

Table 1 bounds the specific components used for this pipeline framework.

Table 1. Hardware and DAQ Software Requirements

Classification	Required Tooling/Resources
Compute System	Arduino Microcontroller (Uno or equivalent), Standard USB A-to-B Serial bus.
Analog Synthesis	Prototyping Breadboard, Basic Resistor/Capacitor kits, Jumper routing wires.
Bench Instrumentation	High-resolution Digital Oscilloscope or ADALM2000 hardware suite.
IDE & Processing	Arduino Studio IDE (C++ deployment), MATLAB (Serial Acquisition Tooling).

Table 2. Visual Reference of Key Equipment

Arduino Uno	**USB cable**	**Digital Oscilloscope**
		
Breadboard	**Resistor/Capacitor Kits**	**MATLAB Interface**
		

6 Laboratory Procedure

6.1 Phase 1: Hardware Assembly and Basic Code Verification

6.1.1 Step 1.1: Design and Assemble RC Filter

Checkpoint (1/7): Implement the Anti-Imaging Filter:Using your Pre-Lab calculations, construct the first-order RC low-pass filter on the breadboard using the specified resistor and capacitor values.

Procedure: Build RC filter using Pre-Lab values. Measure actual components with multimeter. Include photo in lab report.

Deliverable (1/14): Submit breadboard photo with labeled resistor, capacitor, Arduino pin connections, and measured component values.

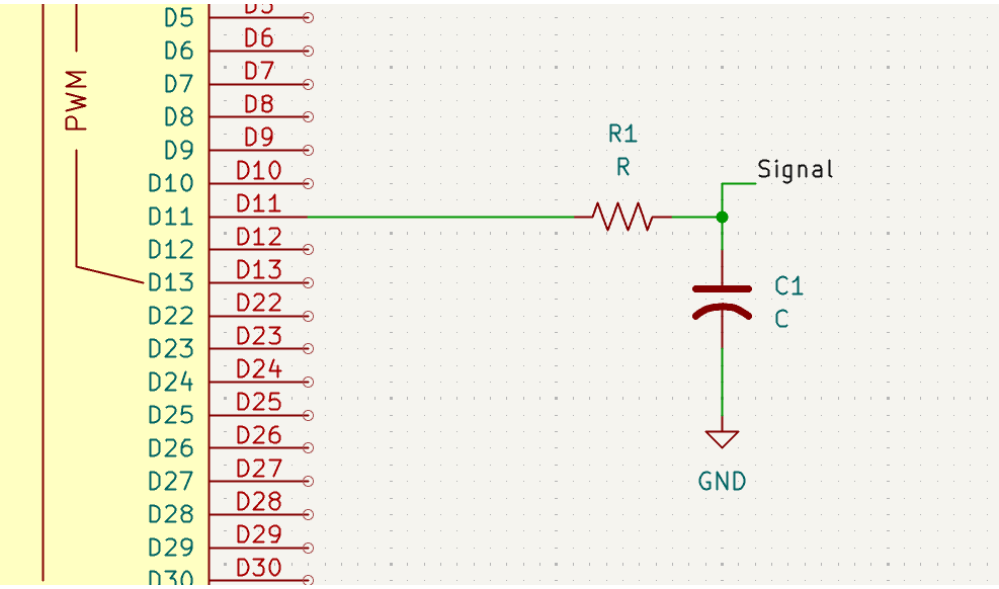


Figure 1. **REFERENCE SCHEMATIC:** Include breadboard photo and measured component values in lab report.

Deliverable (2/14): Your Circuit Documentation: Include breadboard photo with labeled resistor, capacitor, and Arduino connections in the lab report.

6.1.2 Step 1.2: Baseband Verification Test

Checkpoint (2/7): Baseband Verification:Verify that the filter produces expected DC voltage levels corresponding to PWM duty cycles.

Procedure: Test PWM at 0%, 50%, and 100% duty cycles using DMM. Verify filter output is linear (0V $\approx$ 2.5V $\approx$ 5V). Document results.

Deliverable (3/14): Submit DMM verification evidence for 0%, 50%, and 100% duty-cycle measurements.

Example Arduino Code:

```
pinMode(3, OUTPUT);
analogWrite(3, 0);    // 0% duty, measure output
delay(500);
analogWrite(3, 127);  // 50% duty, measure output
delay(500);
analogWrite(3, 255);  // 100% duty, measure output
```

Deliverable (4/14): Your DMM Photo: Photograph your digital multimeter display showing the voltage reading at one of the duty cycle settings (0%, 50%, or 100%). Show your Arduino code on screen if possible, or write it next to the photo.

Discussion (1/7): Compare expected vs. measured DMM voltages at 0%, 50%, and 100% duty cycle. Explain any nonlinearity or offsets using component tolerance, PWM ripple, or measurement setup.

6.2 Phase 2: Signal Acquisition and MATLAB Integration

6.2.1 Step 2.1: Waveform Generation

Checkpoint (3/7): Waveform Generation Matrix:Extend your Arduino code to generate sine, square, and triangle waveforms at your assigned frequency.

Deliverable (5/14): Submit your working waveform-generation code (LUT/algorithmic/hybrid) with timing parameter documentation.

Your Assigned Parameters:

Implementation approaches: (Choose one or more)

- **Look-Up Table (LUT):** Pre-compute 256 values stored in PROGMEM
- **Algorithmic:** Real-time computation using `sin()`, `cos()`, or PWM math
- **Hybrid:** Combine both methods for performance

REFERENCE - Example Sine Wave Implementation:

```
// Method 1: Look-Up Table (LUT)
const byte sine_lut[256] PROGMEM = {128, 131, 134, ...}; // 256 values
void setup() { /* initialize */ }
void loop() {
  for(int i = 0; i < 256; i++) {
    byte val = pgm_read_byte(&sine_lut[i]);
    analogWrite(PWM_PIN, val);
    delayMicroseconds(period_us);
  }
}

// Method 2: Real-time Algorithm
void loop() {
  for(int i = 0; i < 256; i++) {
    float sine_val = 128 + 127*sin(2*PI*i/256);
    analogWrite(PWM_PIN, (byte)sine_val);
    delayMicroseconds(period_us);
  }
}
```

Key Parameters: $T_s = \frac{1}{256 \times (\text{frequency in Hz})}$
For $f_0 = 10$ Hz: $T_s \approx 390$ μs per sample

Figure 2. **REFERENCE IMPLEMENTATIONS:** Include your working waveform generation code in lab report with timing documentation.

6.2.2 Step 2.2: Oscilloscope Capture and Verification

Checkpoint (4/7): Oscilloscope Bench Capture:Connect the filter output to the oscilloscope and capture your waveform.

Procedure:

Capture 3-4 complete periods showing clear waveform with stable trigger. Record time/div and volts/div settings. Photograph screen showing frequency, amplitude, and any distortion.

Deliverable (6/14): Submit oscilloscope screenshot/photo with visible time/div, volts/div, measured frequency, and waveform stability.

REFERENCE - Oscilloscope Display Layout:

Scope Capture Here

Key Measurements to Record:

- Read **Frequency** using vertical reference lines to measure period T
- Measure **Peak-to-Peak voltage** by counting vertical divisions
- Verify trigger is stable (no drift or jitter in waveform display)

Figure 3. **REFERENCE MEASUREMENT GUIDE:** Include scope photo in lab report with frequency, amplitude, and stability assessed.

6.2.3 Step 2.3: MATLAB Serial Data Acquisition

Checkpoint (5/7): MATLAB Serial Cross-Domain Pipeline:Acquire 500 samples of your waveform through serial communication.

Procedure:

Configure Arduino for serial transmission (115200 baud). Write MATLAB script to acquire 500 contiguous samples. Calculate actual sampling period T_s and frequency $f_s = 1/T_s$. Document results.

Deliverable (7/14): Submit MATLAB serial-acquisition script and Arduino transmitter code evidence (screenshots or code snippets).

REFERENCE - MATLAB Serial Communication Template:

```
% Configure serial port
clear; clc;
port = 'COM3';          % Change to your port
baud = 115200;
timeout = 60;           % seconds

% Create serial object
s = serialport(port, baud);
configureTerminator(s, "CR/LF");

% Read data
N_samples = 500;
data = [];
tic;
while length(data) < N_samples
    if s.NumBytesAvailable > 0
        val = readline(s);
        data = [data; str2double(val)];
    end
    if toc > timeout
        disp('Timeout!');
        break;
    end
end

% Process data
Ts = 1/Fs;  % sampling period (calculate from timing)
t = (0:length(data)-1) * Ts;
V = (data / 1023) * 5;  % convert ADC to volts

% Plot
figure; plot(t, V, 'b', 'LineWidth', 1.5);
xlabel('Time (s)'); ylabel('Voltage (V)');
title('Serial Acquisition at YOUR_FREQ Hz');
grid on;
```

Your Arduino Code (Serial Transmission):

```
void setup() { Serial.begin(115200); }
void loop() {
  int adc_val = analogRead(A0);
  Serial.println(adc_val);
  delayMicroseconds(period_us);
}
```


Figure 4. **REFERENCE CODE TEMPLATES:** Include screenshots of your working Serial.read() and analogRead() code in lab report.

6.2.4 Step 2.4: Data Visualization

Deliverable (8/14): Plot the buffered vectors:Create publication-quality plots of your acquired data.

Procedure: Create time vector: $t = [0 : T_s : (N-1) \cdot T_s]$ where $N = 500$. Scale ADC to voltage: $V = \frac{\text{ADC}}{1023} \times 5\text{V}$. Plot with labels, grid, and title. Include screenshot in lab report.

REFERENCE - Expected MATLAB Plot Structure:

```
Good Plot Example Output:

y (Voltage in V)
5.0 |     /\            /\
    |    /  \          /  \
2.5 +   /    \        /    \
    |  /      \      /      \
0.0 +-/--------\----/--------\-
    |           \/            \/
-2.5|
    |
    +--+--+--+--+--+--+--+--+--+---> t (Time in seconds)
    0 0.025 0.05 0.075 0.1 0.125 0.15

Title: "10 Hz Sine Wave - Serial Acquisition (500 samples)"
Legend: "ADC Data from Arduino"
Grid: ON (both x and y)
```

MATLAB Plotting Code:

```
figure('Position', [100 100 800 400]);
plot(t, V, 'b-', 'LineWidth', 1.5);
xlabel('Time (s)', 'FontSize', 12);
ylabel('Voltage (V)', 'FontSize', 12);
title(['Serial Data: ' num2str(measured_freq) ' Hz Sine Wave'], ...
    'FontSize', 14);
grid on;
legend('Arduino ADC Reading', 'Location', 'northeast');
ylim([0 5]);
```

Figure 5. **REFERENCE PLOT STRUCTURE:** Include screenshot of your MATLAB figure (500 samples) with time, voltage, title, and grid in lab report.

Discussion (2/7): Compare your scope waveform and MATLAB waveform for frequency, am-

plitude, and distortion. Identify the dominant source of mismatch between analog capture and serially sampled data.

6.3 Phase 3 (The Challenge): Interrupt Optimization

6.3.1 Step 3.1: Software-Based (Baseline) Assessment

Before implementing interrupt-driven sampling, document the jitter characteristics of standard delay-based code:

Procedure:

Record loop timestamps using `micros()`. Calculate inter-sample intervals and compute mean, std dev, min, max. Document in lab report with histogram.

Deliverable (9/14): Submit baseline (delay-based) jitter histogram and summary statistics (mean, std dev, min, max).

REFERENCE - Jitter Measurement Code (Baseline):

```
// Arduino: Measure delay() loop jitter
void loop() {
  static unsigned long prev_time = 0;
  unsigned long curr_time = micros();

  if (prev_time != 0) {
    unsigned long interval = curr_time - prev_time;
    Serial.println(interval);  // Send to PC
  }

  prev_time = curr_time;
  analogWrite(PWM_PIN, next_value);
  delay(10);  // or whatever period needed
}
```

MATLAB: Histogram of Inter-Sample Intervals

```
% Read intervals from Arduino serial, then:
figure;
histogram(intervals_us, 50, 'FaceColor', [0.3 0.6 0.8]);
xlabel('Inter-Sample Interval (us)');
ylabel('Frequency (counts)');
title('Phase 2 Baseline: Delay-Based Jitter Analysis');
grid on;

% Statistics
mean_interval = mean(intervals_us);
std_interval = std(intervals_us);
fprintf('Mean: %.1f us, Std Dev: %.1f us\n', ...
    mean_interval, std_interval);
```

Expected Result: Histogram shows wide distribution (high jitter) around target interval, with standard deviation typically 5-50

Figure 6. **REFERENCE MEASUREMENT CODE:** Include histogram of inter-sample intervals (Phase 2 baseline) in lab report showing wide distribution.

6.3.2 Step 3.2: Hardware Interrupt Refactoring

Checkpoint (6/7): Hardware Interrupt Refactoring:Implement timer-driven sampling using hardware interrupts.

Procedure: Configure Timer1 for 2× Phase 2 sampling rate. Implement Interrupt Service Routine (ISR) for ADC reads. Remove all `delay()` calls from sampling. Verify waveform quality. Include code in lab report.

Deliverable (10/14): Submit interrupt-driven implementation code (timer setup + ISR) with final sampling-rate settings.

REFERENCE - Timer1 Interrupt Implementation (Arduino Uno):

```
// Global variables
volatile byte sample_index = 0;
const byte sine_lut[256] PROGMEM = { /* 256 values */ };

void setup() {
  Serial.begin(115200);
  pinMode(3, OUTPUT);

  // Configure Timer1 for interrupt-driven sampling
  noInterrupts(); // Disable interrupts

  TCCR1A = 0;
  TCCR1B = 0;
  TCNT1 = 0;

  // Set compare value (adjust for desired frequency)
  // For 10 Hz with 256 samples: OCR1A ~= F_CPU / (256*10*prescale)
  OCR1A = 6249;  // Example for 10 Hz, prescaler=8

  TCCR1B |= (1 << WGM12);    // CTC mode
  TCCR1B |= (1 << CS11);     // Prescaler=8
  TIMSK1 |= (1 << OCIE1A);   // Enable Compare A interrupt

  interrupts(); // Enable interrupts
}

// Interrupt Service Routine (ISR)
ISR(TIMER1_COMPA_vect) {
  byte val = pgm_read_byte(&sine_lut[sample_index]);
  analogWrite(3, val);
  sample_index = (sample_index + 1) % 256;
}

void loop() {
  // Main code - NO sampling here, ISR handles it!
  Serial.println("Interrupt-driven sampling active");
  delay(1000);
}
```

Key Points:

- ISR must be SHORT (microseconds execution)
- Use `volatile` for variables shared between ISR and main code
- Prescaler and OCR1A determine interrupt frequency
- NO `delay()` or `Serial` calls inside ISR

Figure 7. **REFERENCE TIMER1 CONFIGURATION:** Include your ISR implementation (2× frequency) in lab report with register settings explained

6.3.3 Step 3.3: Sampling Rate Augmentation

Checkpoint (7/7): Sampling Rate Augmentation:Double your sampling frequency using the interrupt mechanism.

Deliverable (11/14): Submit Phase 2 vs. Phase 3 jitter comparison (histograms) and computed improvement percentage.

Original Parameters (Phase 2):

Record Phase 2 baseline ($f_{s,\text{orig}}$, $T_{s,\text{orig}}$). Double frequency to $f_{s,\text{new}} = 2 \times f_{s,\text{orig}}$. Measure Phase 3 jitter (mean, std dev, min, max intervals). Calculate improvement factor. Document all results.

REFERENCE - Jitter Comparison Concept:

Histogram Comparison Here

MATLAB Code to Generate Comparison:

```
% Assuming intervals_phase2 and intervals_phase3 measured
figure;

% Compare histograms
subplot(1,2,1);
histogram(intervals_phase2, 50, 'FaceColor', [1 0.5 0]);
xlabel('Interval (us)'); title('Phase 2: Delay-Based Jitter');
xlim([min(intervals_phase2)*0.9, max(intervals_phase2)*1.1]);

subplot(1,2,2);
histogram(intervals_phase3, 50, 'FaceColor', [0 0.7 1]);
xlabel('Interval (us)'); title('Phase 3: Interrupt-Driven Jitter');
xlim([min(intervals_phase2)*0.9, max(intervals_phase2)*1.1]);

% Calculate improvement
jitter_phase2 = std(intervals_phase2);
jitter_phase3 = std(intervals_phase3);
improvement = (jitter_phase2 - jitter_phase3) / jitter_phase2 * 100;
fprintf('Jitter Improvement: %.1f%%\n', improvement);
```

Figure 8. **REFERENCE COMPARISON CODE:** Include side-by-side histograms (Phase 2 vs Phase 3 jitter) in lab report showing improvement percentage.

Discussion (3/7): Explain why timer-interrupt scheduling reduces jitter compared to delay-based loops. Discuss at least one tradeoff (complexity, ISR constraints, or resource usage).

7 Data Analysis

Complete verification of continuous representation using the captured discrete mapping pairs.

7.1 Analysis Step 1: Amplitude Verification

Discussion (4/7): Analog Authenticity Verification:Compare expected versus measured amplitudes across all three capture domains.

Theoretical vs. Experimental Comparison:

Discussion (5/7): Amplitude Verification:Compare theoretical vs. measured (scope and MATLAB): Calculate percent error $\epsilon = \frac{|A_{\text{theory}} - A_{\text{meas}}|}{A_{\text{theory}}} \times 100\%$ for both domains. Explain major discrepancies.

7.2 Analysis Step 2: Frequency Verification

Discussion (6/7): Frequency Verification:Compare programmed f_0 vs. measured (scope and MATLAB). Calculate frequency error: $\epsilon_f = \frac{|f_0 - f_{\text{meas}}|}{f_0} \times 100\%$. Expected $< 10\%$.

7.3 Analysis Step 3: Jitter Impact Assessment

Discussion (7/7): Jitter Impact:Compare Phase 2 (software delay-based) vs. Phase 3 (interrupt-driven) jitter measurements. Calculate reduction percentage: $R = \frac{\sigma_2 - \sigma_3}{\sigma_2} \times 100\%$. Target $30 - 80\%$ improvement.

7.4 Analysis Step 4: Digital-Analog Mapping Proof

Deliverable (12/14): Digital-Analog Mapping (LLM-Proof Integrity Requirement):Provide dual-platform evidence of authentic waveform acquisition.

Required Documentation:

Deliverable (13/14): Proof Documentation:Submit as separate JPG/PNG files in lab report:

- **Scope Photo:** Clear waveform, **handwritten name + ID visible**, time/div, volts/div settings shown
- **MATLAB Plot:** 500-sample time series, proper axes (time in seconds, voltage 0-5V), title with frequency, grid on
- Verify scope and MATLAB show same waveform characteristics (frequency, amplitude)

7.5 Analysis Step 5: Theoretical vs. Systemic Losses Summary Table

Deliverable (14/14): Submit summary table with theoretical vs. measured values and ranked error sources with justification.

Create summary table in lab report showing theoretical vs. measured values with percent error for amplitude, frequency, and period. Identify primary error sources: component tolerances, ADC quantization, PWM ripple, jitter, signal losses. Prioritize errors by magnitude.

8 Grading Rubric

Total Points: 100

Table 3. Detailed Grading Rubric with Specific Criteria

Category	Evaluation Criteria	Weight
Pre-Lab (15 pts)	Q1: RC filter values w/ f_c calculation ($\pm 10\%$)	5
	Q2: Impulse response $h(t)$ and $\tau = RC$	5
	Q3: Quantization ΔV and SQNR formula	5
Hardware (12 pts)	RC filter constructed & verified ($\leq 5\%$ error)	6
	DMM photos at 0%, 50%, 100% duty cycles	6
Scope Capture (10 pts)	Photo: 3-4 periods, frequency/amplitude $\leq$ 10% error	5
	Visible settings (time/div, volts/div), stable trigger	5
MATLAB Acquisition (10 pts)	500 samples acquired, time/voltage axes correct	5
	Amplitude within 10% of scope, grid & title included	5
Phase 3 Interrupt (12 pts)	ISR implemented, 2× frequency achieved	6
	Jitter reduction $\geq 30\%$, blocking calls removed	6
Proof Documentation (21 pts)	Scope photo: legible name/ID, waveform clear, settings visible	10
	MATLAB plot: time/voltage axes, grid, title, matches scope	11
Analysis & Errors (20 pts)	Error calculations & comparison table (amplitude, freq, period)	10
	Identification & ranking of 3+ error sources with justification	10
Totals		**100**